

May 1st, 2025

Fumiya Kawase

fkawase@berklee.edu

Course: MTEC-347

Sharing is Caring

I developed a MIDI-controlled electronic music performance system featuring four primary synth instruments: bass, kick, synth pad, and snare. The system was designed using the Akai MIDIMIX Controller, but it is compatible with any MIDI controller that supports the corresponding MIDI numbers (see Picture 1 and Table 1). In Picture 1, I configured all controls on Channel 2, but the channel number does not affect performance since the code is channel-agnostic. The code consists of three main sections: Initial Setup, MIDI Mapping and Synth Definitions.

Table 1

Volume (Bass)	CC#1
Volume (Kick)	CC#2
Volume (Synth1)	CC#3
Volume (Snare)	CC#4
Bass Tempo	CC#10
Bass Range	CC#11
Bass Low Pass Filter	CC#12
Kick Density	CC#13
Kick Decay	CC#14
Kick Pitch	CC#15
Synth1 Frequency	CC#16
Synth1 Resonance	CC#17
Synth1 Decay	CC#18
Snare Decay	CC#19
Snare Delay Time	CC#20
Snare Reverb	CC#21
Bass On/Off	Note#1
Kick On/Off	Note#2
Synth1 On/Off	Note#3
Snare On/Off	Note#4

```

// Boot server and initialize MIDI

s.boot;

MIDIClient.init;

MIDIIn.connectAll;


//MIDI Mapping

s.waitForBoot {

  // Create a MIDI control bus for volume

    // Bass

    ~midiVolumeBus1 = Bus.control(s, 1).set(0); // Default volume 0

    MIDIDef.cc(\volumeControl1, { |val|

      var normVal = val / 127; // Normalize MIDI Value

      var expVal = pow(normVal, 3); // Cubic curve for smoother fade

      ~midiVolumeBus1.set(expVal);

      ("Volume: " ++ expVal.round(0.01)).postln; // Print value

    }, ccNum: 1); // Listen to CC#1


    //Kick

    ~midiVolumeBus2 = Bus.control(s, 1).set(0);

    MIDIDef.cc(\volumeControl2, { |val|

```

```

var normVal = val / 127;

var expVal = pow(normVal, 3);

~midiVolumeBus2.set(expVal);

("Volume: " ++ expVal.round(0.01)).postln;

}, ccNum: 2);

//Synth Pad

~midiVolumeBus3 = Bus.control(s, 1).set(0);

MIDIdef.cc(\volumeControl3, { |val|

var normVal = val / 127;

var expVal = pow(normVal, 3);

~midiVolumeBus3.set(expVal);

("Volume: " ++ expVal.round(0.01)).postln;

}, ccNum: 3);

//Snare

~midiVolumeBus4 = Bus.control(s, 1).set(0);

MIDIdef.cc(\volumeControl4, { |val|

var normVal = val / 127;

var expVal = pow(normVal, 3);

~midiVolumeBus4.set(expVal);

("Volume: " ++ expVal.round(0.01)).postln;

}, ccNum: 4);

// Create a control buses

//Bass

```

```

//Tempo

~tempoBus = Bus.control(s, 1).set(0.5); // Default tempo: 0.5 Hz

MIDDef.cc(\tempoControl, { |val|

    var mappedTempo = val.linexp(0, 127, 0.5, 4.0); // Map 0-127 to 0.5-4.0

    ~tempoBus.set(mappedTempo);

    ("Tempo: " ++ mappedTempo.round(0.01) ++ " Hz").postln;

}, ccNum: 10); // Listen to CC#10

```

```

//Range

~bassRangeBus = Bus.control(s, 1).set(3);

MIDDef.cc(\bassRangeControl, { |val|

    var mappedRange = val.linexp(0, 127, 1, 20);

    ~bassRangeBus.set(mappedRange);

    ("BassRange: " ++ mappedRange.round(0.1) ++ " semitones").postln;

}, ccNum: 11);

```

```

//Low Pass Filter

~bassLPFBus = Bus.control(s, 1).set(5);

MIDDef.cc(\bassLPF, { |val|

    var bassLPF = val.linexp(0, 127, 1, 12);

    ~bassLPFBus.set(bassLPF);

    ("BassLPF: " ++ bassLPF.round(0.1) ++ " multiplier").postln;

}, ccNum: 12);

```

```

//kick

```

```

//Frequency

```

```
~kickDensityBus = Bus.control(s, 1).set(4);

MIDDef.cc(\kickDensity, { |val|

    var kickDensity = val.linexp(0, 127, 1, 50);

~kickDensityBus.set(kickDensity);

("kickDensity: " ++ kickDensity.round(0.1) ++ " Hz").postln;

}, ccNum: 13);
```

```
//Decay

~kickDecayBus = Bus.control(s, 1).set(0.5);

MIDDef.cc(\kickDecay, { |val|

    var kickDecay = val.linexp(0, 127, 0.1, 1);

~kickDecayBus.set(kickDecay);

("kickDecay: " ++ kickDecay.round(0.001) ++ " ms").postln;

}, ccNum: 14);
```

```
//Pitch

~kickPitchBus = Bus.control(s, 1).set(1);

MIDDef.cc(\kickPitch, { |val|

    var kickPitch = val.linexp(0, 127, 1, 2);

~kickPitchBus.set(kickPitch);

("kickPitch: " ++ kickPitch.round(0.1) ++ "").postln;

}, ccNum: 15);
```

```
//Synth Pad
```

```
//Low Pass Filter
```

```
~synth1FreqBus = Bus.control(s, 1).set(20000);
```

```
MIDDef.cc(\synth1Freq, { |val|  
    var synth1Freq = val.linexp(0, 127, 20, 20000);  
    ~synth1FreqBus.set(synth1Freq);  
    ("synth1Freq: " ++ synth1Freq.round(0.01) ++ " Hz").postln;  
}, ccNum: 16);
```

```
//Resonance  
~synth1ResBus = Bus.control(s, 1).set(0.3);  
MIDDef.cc(\synth1Res, { |val|  
    var synth1Res = val.linexp(0, 127, 1, 0.01);  
    ~synth1ResBus.set(synth1Res);  
    ("synth1Res: " ++ synth1Res.round(0.01) ++ "").postln;  
}, ccNum: 17);
```

```
//Decay  
~synth1DecayBus = Bus.control(s, 1).set(1);  
MIDDef.cc(\synth1Decay, { |val|  
    var synth1Decay = val.linexp(0, 127, 0.1, 1);  
    ~synth1DecayBus.set(synth1Decay);  
    ("synth1Decay: " ++ synth1Decay.round(0.1) ++ " ms").postln;  
}, ccNum: 18);
```

```
//Snare
```

```
//Decay
```

```
~snareDecayBus = Bus.control(s, 1).set(0.1);
```

```
MIDDef.cc(\snareDecay, { |val|
```

```

    var decay = val.linexp(0, 127, 0.05, 1.5);
    ~snareDecayBus.set(decay);
    ("snareDecay: " ++ (decay*1000).round(1) ++ " ms").postln;
}, ccNum: 19);

```

//Delay Time

```

~snareDlytimeBus = Bus.control(s, 1).set(0.1);
MIDDef.cc(\snareDlytime, { |val|
    var snareDlytime = val.linexp(0, 127, 0.01, 0.3);
    ~snareDlytimeBus.set(snareDlytime);
    ("snareDlytime: " ++ snareDlytime.round(0.001)).postln;
}, ccNum: 20);

```

//Reverb

```

~snareReverbBus = Bus.control(s, 1).set(0.1);
MIDDef.cc(\snareReverb, { |val|
    var snareReverb = val.linexp(0, 127, 0.01, 1);
    ~snareReverbBus.set(snareReverb);
    ("snareReverb: " ++ snareReverb.round(0.01)).postln;
}, ccNum: 21);

```

// Define the Synth

//Bass

```

SynthDef(\bass, {
    var steps = 16; // Number of steps

```



```

    var root = 24 + [Scale.minorPentatonic.degrees,
Scale.minorPentatonic.degrees.choose].choose; // Random Root note

    var tempo = In.kr(~tempoBus.index); // Read tempo from bus
    var range1 = In.kr(~bassRangeBus.index); // Read pitch range
    var range2 = In.kr(~bassRangeBus.index) * -1; // Read volume
    var volume = In.kr(~midiVolumeBus1.index); // Read volume
    var freqLPF = In.kr(~bassLPFBus.index); // Read LPF multiplier

    // Bass sequence

    var notes = Array.fill(4, { rrand(range2, range1) + root });

        var seq1 = Demand.ar(Impulse.ar(tempo), 0, Dseq(notes, inf));
        var seq2 = Demand.ar(Impulse.ar(tempo), 0, Dseq(notes + 12, inf));
        var seq3 = Demand.ar(Impulse.ar(tempo), 0, Dseq(notes + 19, inf));

    var freq1 = seq1.midicps;

        var freq2 = seq2.midicps;
        var freq3 = seq3.midicps;

    // Pluck synthesis

    var pluckEnv = EnvGen.ar(Env.perc(0.1, 0.03, -16), Impulse.ar(tempo));
    var exciter = PinkNoise.ar() * pluckEnv;

    var sig1 = Pluck.ar(exciter, Impulse.ar(tempo), freq1.reciprocal, freq1.reciprocal, 4, 0.1);
        var sig2 = Pluck.ar(exciter, Impulse.ar(tempo), freq2.reciprocal, freq2.reciprocal,
4, 0.1);
        var sig3 = Pluck.ar(exciter, Impulse.ar(tempo), freq3.reciprocal, freq3.reciprocal,
4, 0.1);

```

```

// Distortion and filtering

    var bass = (RLPF.ar(sig1, freq1 * freqLPF, 0.2) + RLPF.ar(sig2, freq2 * freqLPF,
0.2) + RLPF.ar(sig3, freq3 * freqLPF, 0.2)).distort;

    Out.ar(0, bass * volume ! 2);

}).add;

//Kick

SynthDef(\kick, {

    var volume = In.kr(~midiVolumeBus2.index);

    var tempo = LFNoise2.kr({0.01.rand}.dup(2),
[0.01.rand,0.001.rand,0.5.rand].choose, 1.0); // Random timing variation

    var decay = In.kr(~kickDecayBus.index); // Read decay time
    var time = min(1/tempo, decay);

    var range1 = In.kr(~kickDensityBus.index); //Read kick frequency
    var density = LFNoise2.kr(0.1).range(1, range1);

    var trig = Dust.kr(density); // Random triggers
    var pitch = In.kr(~kickPitchBus.index); // Read pitch

    //Sub-bass + click

    var subFreq = Demand.kr(trig, 0, Drand([40, 38, 42, 45] * pitch, inf));
    var clickFreq = Demand.kr(trig, 0, Dwhite(6000, 10000, inf));

    // Envelope

    var kickenv = EnvGen.kr(Env.perc(0.001, time, -4), trig);

```

```

// Sound generation

var sub = SinOsc.ar(subFreq).tanh * kickenv;

var click = BPF.ar(WhiteNoise.ar(), clickFreq, 0.2) * EnvGen.kr(Env.perc(0.001,
0.01), trig);

var kick = (sub + (click * 0.01)).tanh * volume;

Out.ar(0, kick ! 2);

}).add;

```

//Synth Pad

```

SynthDef(\synth1, {

    var volume = In.kr(~midiVolumeBus3.index);

    var density = LFNoise2.kr(0.1).range(5, 20); // Random trigger rate

    var trig = Dust.kr(density); // Random triggers

    var freq = In.kr(~synth1FreqBus.index); // Filter cutoff

    var res = In.kr(~synth1ResBus.index); // Resonance

    var decay = In.kr(~synth1DecayBus.index); // Decay time

    // Envelope

    var env = EnvGen.kr(Env.perc(0.01, 0.1, -4), trig);

    // Sound generation

    var raw = PinkNoise.ar() * env * 3;

    var filtered = RLPF.ar(raw, freq, res); // Low-pass filter

    // Effects

```

```
2.do { |i| filtered = AllpassC.ar(filtered, 0.2, Rand(0.01, 0.05), decay * Rand(0.5,  
1.5) ) * 0.8 + filtered;};
```

```
filtered = (filtered * 2).tanh * 0.8;
```

```
Out.ar(0, filtered * volume ! 2);
```

```
}).add;
```

```
// Snare
```

```
SynthDef(\snare, {
```

```
var volume = In.kr(~midiVolumeBus4.index);
```

```
var decay = In.kr(~snareDecayBus.index); // Read decay
```

```
var dlyTime = In.kr(~snareDlytimeBus.index); // Read delay time
```

```
var reverbAmt = In.kr(~snareReverbBus.index); // Read Reverb amount
```

```
var trig = Dust.kr(3); // Random triggers
```

```
//Envelope
```

```
var env = EnvGen.kr(Env.perc(0.001, decay), trig);
```

```
//Sound generation
```

```
var noise = WhiteNoise.ar() * env;
```

```
//Effects
```

```
var delayed = CombC.ar(noise, 0.3, dlyTime, decay); // Delay
```

```
var wet = FreeVerb.ar(delayed, reverbAmt, 0.7); // Reverb
```

```
var snare = (noise + wet).tanh * volume; //Mix & clip
```

```

        Out.ar(0, snare ! 2);
    }).add;

s.sync; // Wait for SynthDef to be added

// Toggle the synth with MIDI notes
~bass = Synth(\bass);
~kick = Synth(\kick);
    ~synth1 = Synth(\synth1);
    ~snare = Synth(\snare);

MIDIn.noteOn(\noteToggle, { |vel, note|
    if (note == 1) {
        if (~bass.isNil) {
            ~bass = Synth(\bass);
            "Bass STARTED".postln;
        } {
            ~bass.free;
            ~bass = nil;
            "Bass STOPPED".postln;
        };
    };

    if (note == 2) {
        if (~kick.isNil) {
            ~kick = Synth(\kick);
            "Kick STARTED".postln;
        };
    };
};

```

```

    }{
        ~kick.free;

        ~kick = nil;

        "Kick STOPPED".postln;
    };
};

        if (note == 3) {
if (~synth1.isNil) {
    ~synth1 = Synth(\synth1);

    "Synth1 STARTED".postln;
} {
    ~synth1.free;

    ~synth1 = nil;

    "Synth1 STOPPED".postln;
};
};

        if (note == 4) {
if (~snare.isNil) {
    ~snare = Synth(\snare);

    "Snare STARTED".postln;
} {
    ~snare.free;

    ~snare = nil;

    "Snare STOPPED".postln;
};
};
};

```

});

};